

Q-Learning in Practice

CS 185/285

Instructor: Sergey Levine
UC Berkeley



Part 1:

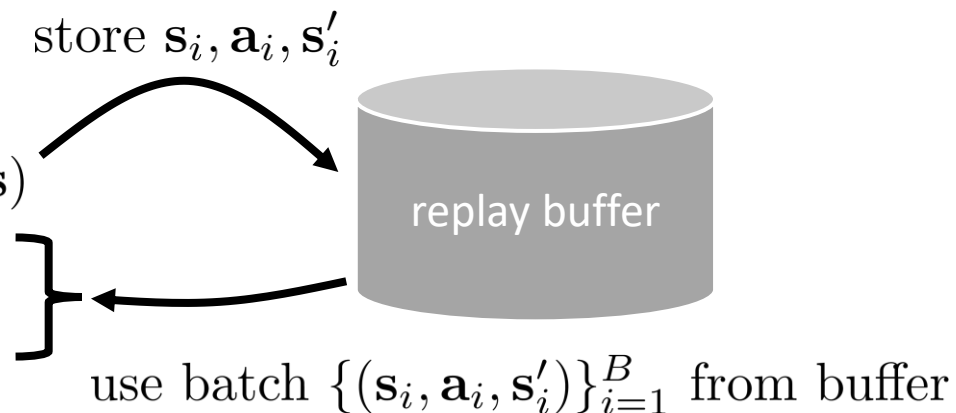
Target networks

From last time

原始的Q-learning每次仅执行单步梯度下降更新参数，更新后网络的Q值函数会发生偏移，而目标值的计算又依赖这个偏移后的函数，这就形成了“边走边改目标”的动态过程。这样容易导致训练过程出现震荡

Q-learning:

- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}'_i, \mathbf{a}')$
 3. update θ using $\nabla_{\theta} \sum_{i=1}^B \|Q_{\theta}(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

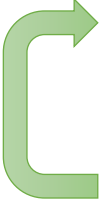


$$\pi_{\theta}(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 - \epsilon & \text{if } \mathbf{a} = \arg \max_{\mathbf{a}} Q_{\theta}(\mathbf{s}, \mathbf{a}) \\ \epsilon / (|\mathcal{A}| - 1) & \text{otherwise} \end{cases}$$

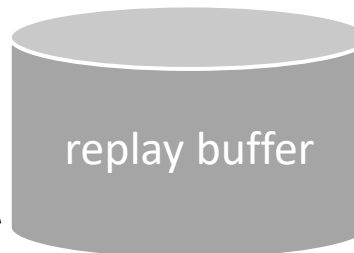
off-policy: $B \geq 1$

What's wrong?

Q-learning:

- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')$
 3. update θ using $\nabla_\theta \sum_{i=1}^B \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

store $\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i$



use batch $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}_{i=1}^B$ from buffer

经验回放缓冲区的作用是打破数据相关性，提升训练稳定性，更新时需从缓冲区采样批量数据

Q-learning is *not* gradient descent!

$$\theta \leftarrow \phi - \alpha \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')])$$

没有正确实现求导的链式法则

no gradient through target value

Q-learning and regression

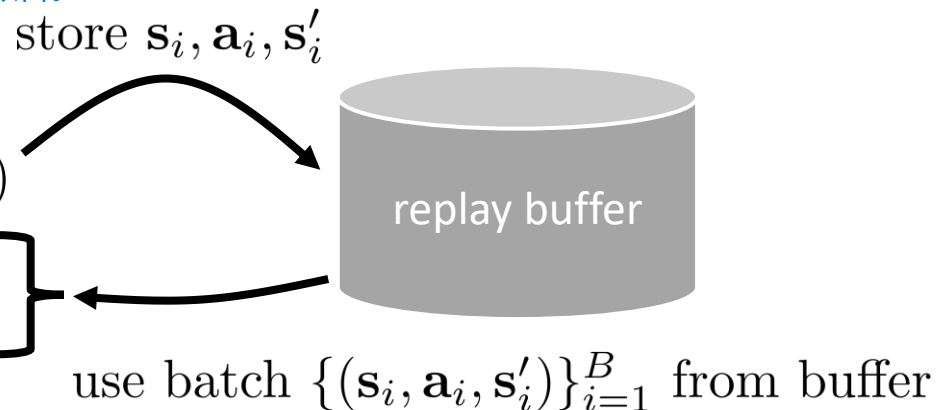
Q-learning是面向强化学习的在线学习过程，目标值随参数动态变化，优化过程存在“移动目标”的特性；而完整拟合Q迭代算法是离线回归任务，目标值固定，优化过程更稳定可控。这种差异也决定了它们在实际应用中的不同场景，Q-learning更适合在线决策的强化学习任务，而完整拟合Q迭代算法更偏向于静态数据集上的回归拟合。

Q-learning:

1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')$
3. update θ using $\nabla_\theta \sum_{i=1}^B \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

one gradient step, moving target

$$\theta \leftarrow \phi - \alpha \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i) (Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')])$$



full fitted Q-iteration algorithm:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')$
3. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

perfectly well-defined, stable regression

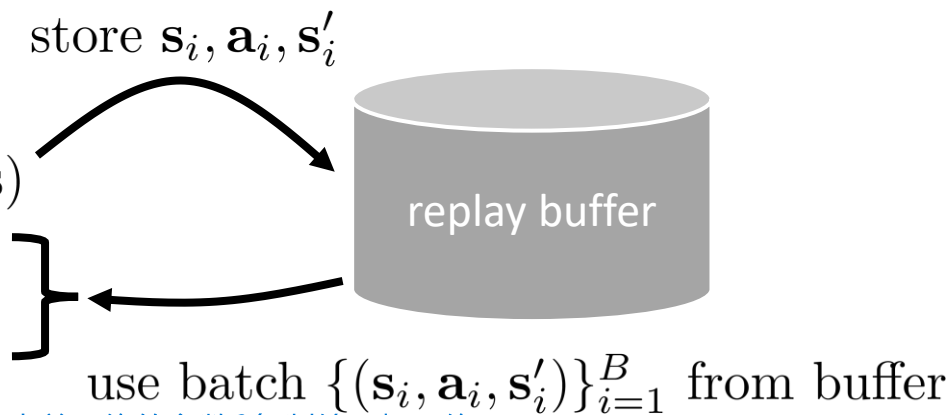
this doesn't change during
“inner” regression SGD loop

$$\theta \leftarrow \phi - \alpha \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i) (Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}')])$$

Q-learning with target networks

Q-learning with target network:

1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}')$
3. update θ using $\nabla_\theta \sum_{i=1}^B \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
4. every 10,000 steps or so, set $\bar{\theta} \leftarrow \theta$ 每隔约10000步, 将当前网络的参数 θ 复制给目标网络



targets only change every 10,000 steps

why is this good?

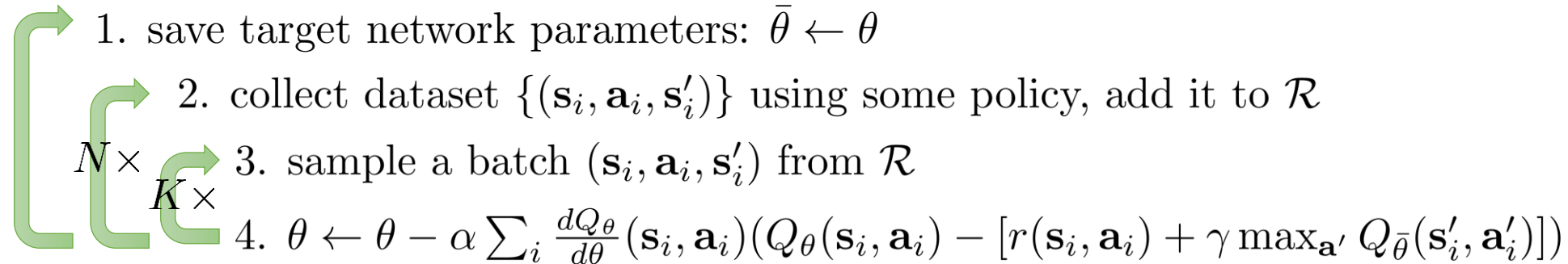
目标网络的参数固定, 在计算目标值时不会随当前网络参数更新而变化, 避免了Q-learning中“移动目标”带来的训练震荡, 让优化过程更稳定
目标网络相当于提供了一个“滞后”的Q值参考, 避免当前网络过度拟合近期的经验数据, 提升泛化能力

why is this bad?

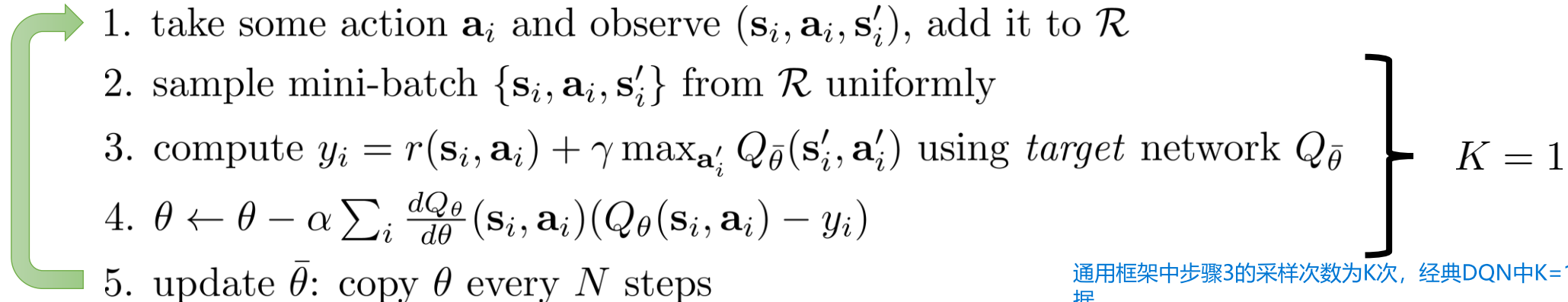
目标网络参数更新间隔较长, 会导致目标值始终滞后于当前网络的最优状态, 可能让算法陷入局部最优, 无法及时利用最新的学习成果

“Classic” deep Q-learning algorithm (DQN)

Generalized Q-learning with replay buffer and target network:

- 
1. save target network parameters: $\bar{\theta} \leftarrow \theta$
 2. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy, add it to $\mathcal{R}$
 3. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ from $\mathcal{R}$
 4. $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)])$

“classic” deep Q-learning algorithm:

- 
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$, add it to $\mathcal{R}$
 2. sample mini-batch $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ from $\mathcal{R}$ uniformly
 3. compute $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)$ using *target* network $Q_{\bar{\theta}}$
 4. $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i)$
 5. update $\bar{\theta}$: copy θ every N steps
- $K = 1$

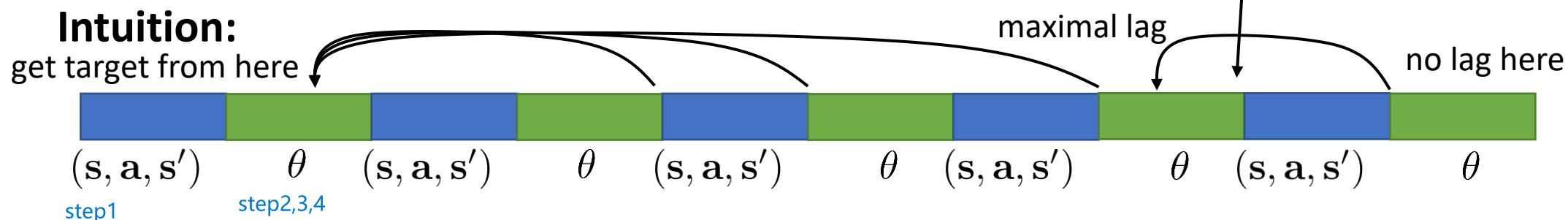
通用框架中步骤3的采样次数为K次，经典DQN中K=1，即每次迭代仅采样一次批量数据。

You'll implement this in HW3!

Alternative target network

“classic” deep Q-learning algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$, add it to $\mathcal{R}$
2. sample mini-batch $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ from $\mathcal{R}$ uniformly
3. compute $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)$ using *target* network $Q_{\bar{\theta}}$
4. $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_{\theta}}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_{\theta}(\mathbf{s}_i, \mathbf{a}_i) - y_i)$
5. update $\bar{\theta}$



Feels weirdly uneven, can we always have the same lag?

Popular alternative (similar to Polyak averaging):

5. update $\bar{\theta}$: $\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$

$\tau = 0.999$ works well

平滑更新目标网络

Part 2:

More efficient Q-learning

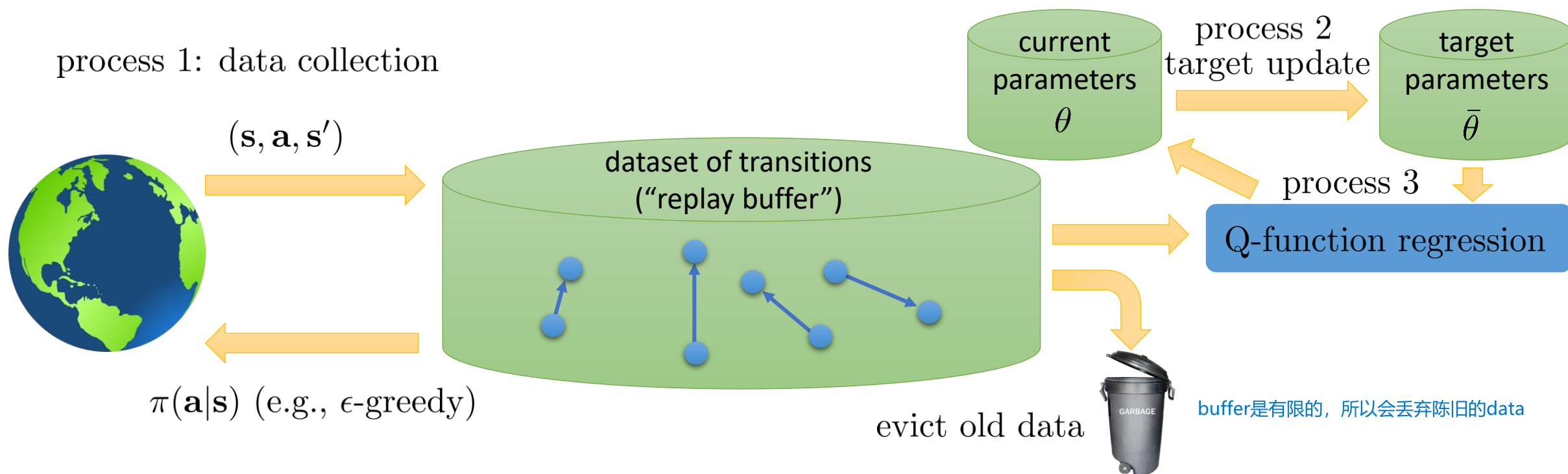
The general view

general conclusion of Q-learning, 所有Q-learning的algorithm都是基于这个框架, 在buffer, 每个process执行的顺序和速度上做微调

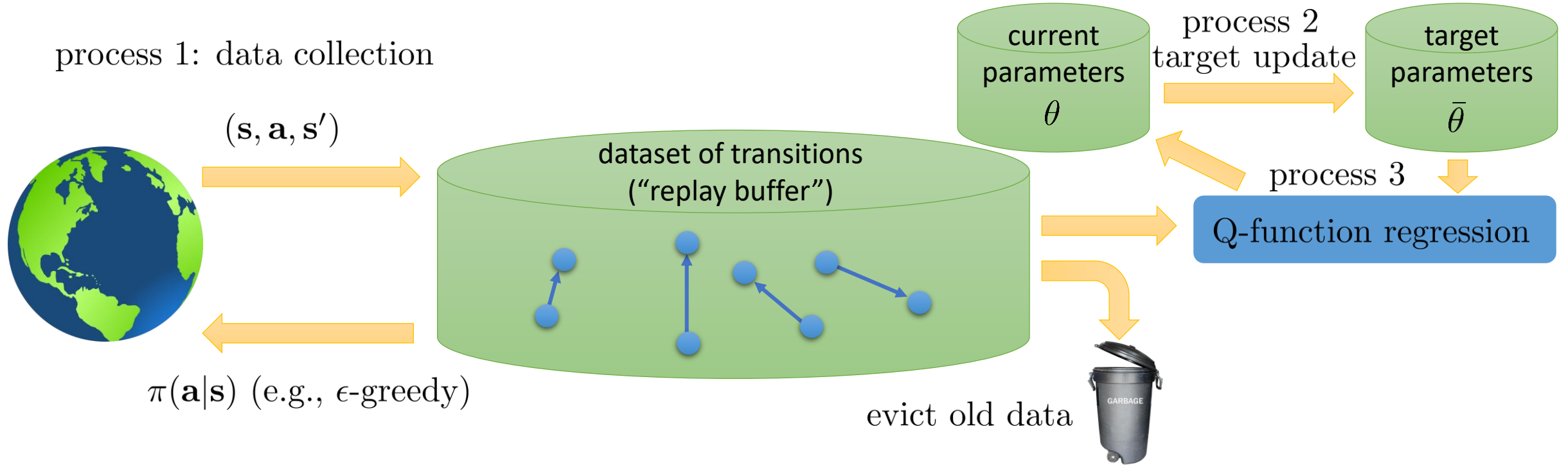
Generalized Q-learning with replay buffer and target network:

DQN: $N = 1, K = 1$

1. save target network parameters: $\bar{\theta} \leftarrow \theta$
2. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy, add it to $\mathcal{R}$
- $N \times$ 3. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ from $\mathcal{R}$
- $K \times$ 4. $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i) (Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)])$



A more general view



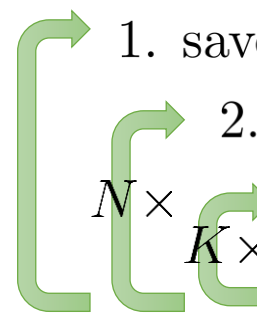
online Q learning 相当于是 buffer == 1

- Online Q-learning (last lecture): evict immediately, process 1, process 2, and process 3 all run at the same speed
- DQN: process 1 and process 3 run at the same speed, process 2 is slow
- Fitted Q-iteration: process 3 in the inner loop of process 2, which is in the inner loop of process 1

Which settings make it more efficient?

Generalized Q-learning with replay buffer and target network:

DQN: $N = 1, K = 1$

- 
- 1. save target network parameters: $\bar{\theta} \leftarrow \theta$
 - 2. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy, add it to $\mathcal{R}$
 - $N \times$ 3. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ from $\mathcal{R}$
 - $K \times$ 4. $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i) (Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)])$

what happens as we increase K ?

sometimes called “update-to-date” (UTD) ratio

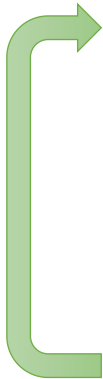
higher UTD = fast learning (but costs more compute)

too high UTD = “overfitting” (we’ll learn about this in the offline RL lectures!)

what happens as we decrease N ?

Value propagation

“classic” deep Q-learning algorithm:

- 
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$, add it to $\mathcal{R}$
 2. sample mini-batch $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ from $\mathcal{R}$ uniformly
 3. compute ~~$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)$~~ using *target* network $Q_{\bar{\theta}}$
 4. $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_{\theta}}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_{\theta}(\mathbf{s}_i, \mathbf{a}_i) - y_i)$
 5. update $\bar{\theta}$: copy θ every $N = 10,000$ steps

S							+1

say $Q_{\bar{\theta}}(\mathbf{s}, \mathbf{a}) = 0$ initially for all $(\mathbf{s}, \mathbf{a})$

how many steps until we propagate +1 to S?

Idea: what if we use n -step returns?

$$y_t^{(i)} = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) + \gamma^n \max_{\mathbf{a}_{t+n}^{(i)}} Q_{\bar{\theta}}(\mathbf{s}_{t+n}^{(i)}, \mathbf{a}_{t+n}^{(i)})$$

前半部分是未来 n 步的累积折扣奖励，后半部分是第 $n+1$ 步之后的状态最大Q值。

价值传播效率更高，能更快将终点的+1奖励传递到起始状态S，减少学习收敛时间。

why is this good?

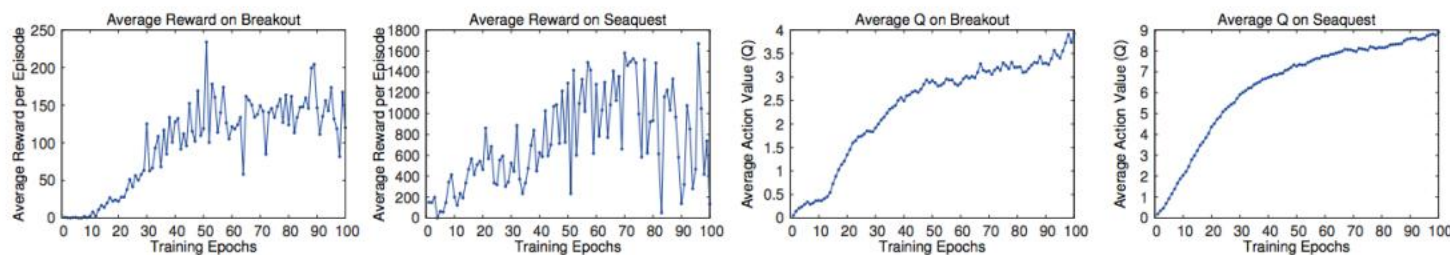
增加了计算复杂度，需要存储和处理更多步的状态动作序列。

why is this bad?

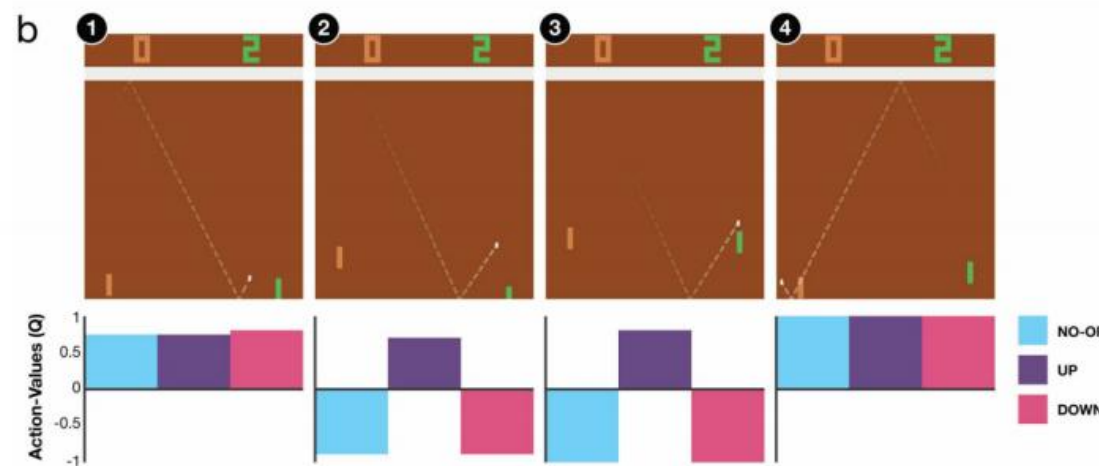
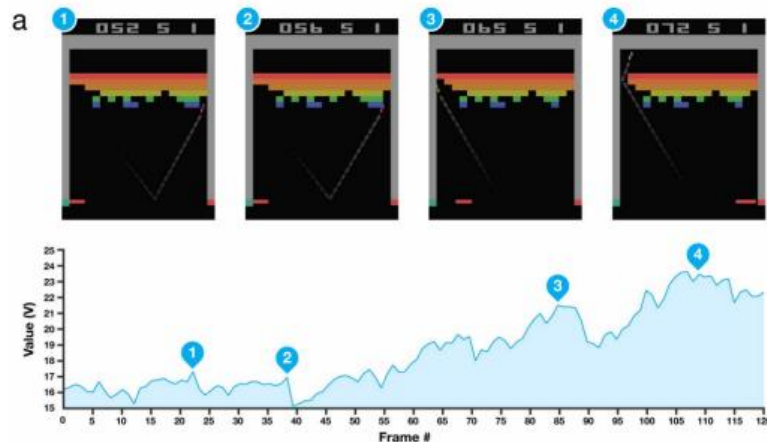
Part 3:

Overestimation in Q-learning

Are the Q-values accurate?

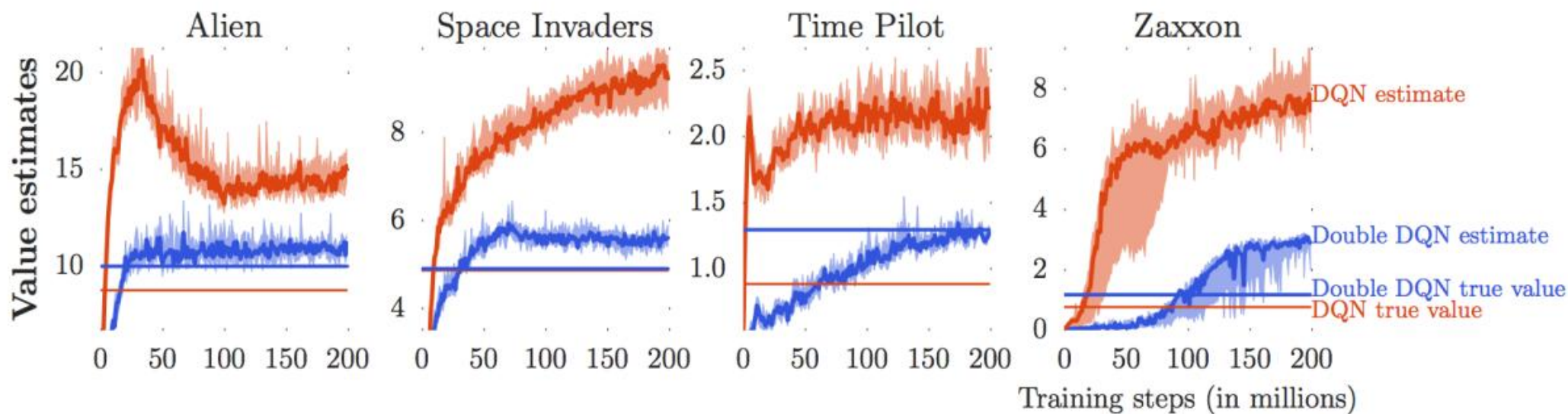


As predicted Q increases, so does the return



Are the Q-values accurate?

通过4组 Atari 游戏的实验数据，验证了 Double DQN 算法的Q值估计准确性远优于传统DQN算法，Double DQN的Q值估计能更贴近真实Q值，而DQN算法存在明显的Q值过估计问题。



Overestimation in Q-learning

target value $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)$

因为Q值估计本身存在噪声，取最大值的操作会放大这种噪声，使得算法高估后续状态的真实价值。

this last term is the problem

imagine we have two random variables: X_1 and X_2

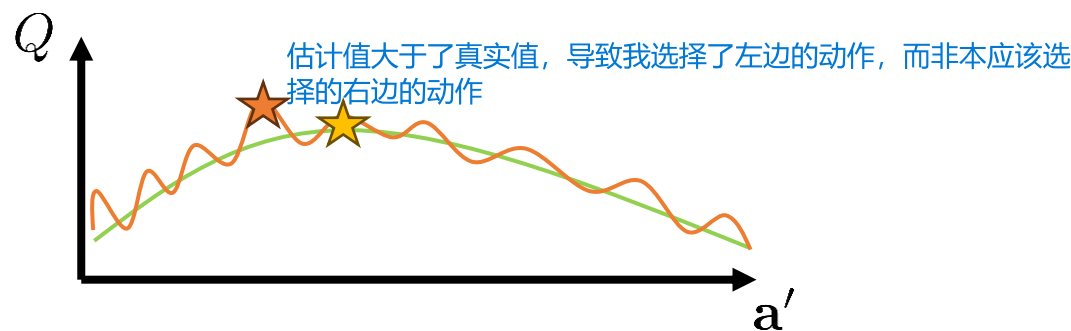
$$E[\max(X_1, X_2)] \geq \max(E[X_1], E[X_2])$$

$Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}')$ is not perfect – it looks “noisy”

hence $\max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}')$ *overestimates* the next value!

note that $\max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}') = \underline{Q_{\bar{\theta}}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}'))}$

value *also* comes from $Q_{\bar{\theta}}$ action selected according to $Q_{\bar{\theta}}$



这种过估计问题会让智能体倾向于选择被高估的动作，可能导致策略偏离最优解。为了解决这个问题，后续衍生出了Double DQN等改进算法，通过分离动作选择和动作评估的机制，有效缓解了Q值过估计的问题。

Double Q-learning

$$E[\max(X_1, X_2)] \geq \max(E[X_1], E[X_2])$$

note that $\max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}') = \underline{Q_{\bar{\theta}}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}'))}$

value *also* comes from $Q_{\bar{\theta}}$ action selected according to $Q_{\bar{\theta}}$

if the noise in these is decorrelated, the problem goes away!

idea: don't use the same network to choose the action and evaluate value!

“double” Q-learning: use two networks: 两个不同的网络，各自的更新方式如下

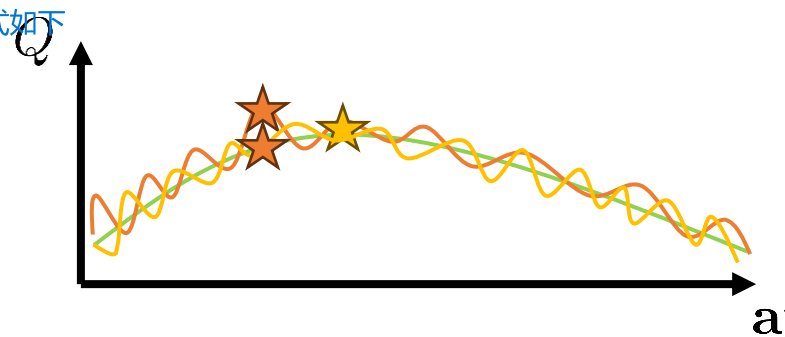
$$\underline{Q_{\theta_A}(\mathbf{s}, \mathbf{a})} \leftarrow r + \gamma Q_{\theta_B}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta_A}(\mathbf{s}', \mathbf{a}'))$$

选择最优动作

$$Q_{\theta_B}(\mathbf{s}, \mathbf{a}) \leftarrow r + \gamma Q_{\theta_A}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta_B}(\mathbf{s}', \mathbf{a}'))$$

评估该动作价值

if the two Q's are noisy in *different* ways, there is no problem



由于两个网络的噪声是相互独立的，当一个网络高估某个动作的价值时，另一个网络的评估会相对客观，最终的Q值估计会趋近于真实值，就像右侧曲线图中，两个网络的估计曲线峰值（星形标记）都更贴近真实Q值曲线，缓解了过估计问题。

Double Q-learning in practice

where to get two Q-functions? just use the current and target networks!

standard Q-learning: $y = r + \gamma Q_{\bar{\theta}}(s', \arg \max_{\mathbf{a}'} Q_{\bar{\theta}}(s', \mathbf{a}'))$

double Q-learning: $y = r + \gamma Q_{\bar{\theta}}(s', \arg \max_{\mathbf{a}'} Q_{\theta}(s', \mathbf{a}'))$ 用当前网络进行动作选择, 目标网络负责价值评估

just use current network (not target network) to evaluate action

still use target network to evaluate value!

这种实现方式无需额外增加网络结构, 复用Q-learning已有的组件, 就能让动作选择和价值评估的噪声相互解耦。

Generalized double Q-learning

double Q-learning: $y = r + \gamma Q_{\bar{\theta}}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}', \mathbf{a}'))$

对所有可能动作的Q值进行加权求和，这种方式能平滑Q值估计，减少单个动作高估带来的影响。


$$\mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}'|\mathbf{s}')} [Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}'))]$$

$$\pi_{\theta}(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 & \text{if } \mathbf{a} = \arg \max Q_{\theta}(\mathbf{s}, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$$

could be anything, including another actor!

“clipped” double Q-learning: $y = r + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}'|\mathbf{s}')} [\underbrace{\min_{i \in \{1,2\}} Q_{\bar{\theta}_i}(\mathbf{s}', \mathbf{a}'))}_{\text{min over an ensemble of (usually 2) target Q-networks}}]$

min over an ensemble of (usually 2) *target* Q-networks

less common with Q-learning, but very often used with Q-function actor-critic

Simple practical tips for Q-learning

- Q-learning takes some care to stabilize
 - Test on easy, reliable tasks first, make sure your implementation is correct

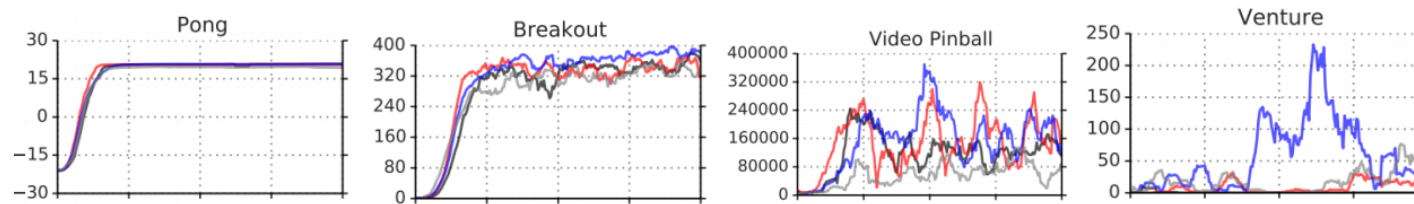


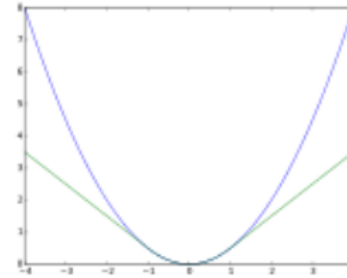
Figure: From T. Schaul, J. Quan, I. Antonoglou, and D. Silver. “Prioritized experience replay”. *arXiv preprint arXiv:1511.05952* (2015), Figure 7

- Large replay buffers help improve stability
 - Looks more like fitted Q-iteration
- It takes time, be patient – might be no better than random for a while
- Start with high exploration (epsilon) and gradually reduce

Advanced tips for Q-learning

- Bellman error gradients can be big; clip gradients or use Huber loss

$$L(x) = \begin{cases} x^2/2 & \text{if } |x| \leq \delta \\ \delta|x| - \delta^2/2 & \text{otherwise} \end{cases}$$



- Double Q-learning helps *a lot* in practice, simple and no downsides
- N-step returns also help a lot, but have some downsides
- Schedule exploration (high to low) and learning rates (high to low), Adam optimizer can help too
- Run multiple random seeds, it's very inconsistent between runs

Intermission

Part 4:

Back to continuous actions

Q-learning with continuous actions

What's the problem with continuous actions?

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q_{\theta}(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

this max

$$\text{target value } y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)$$

this max
particularly problematic (inner loop of training)

How do we perform the max?

Option 1: optimization

- gradient based optimization (e.g., SGD) a bit slow in the inner loop
- action space typically low-dimensional – what about stochastic optimization?

Q-learning with stochastic optimization

Simple solution:

$$\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}) \approx \max \{Q(\mathbf{s}, \mathbf{a}_1), \dots, Q(\mathbf{s}, \mathbf{a}_N)\}$$

$(\mathbf{a}_1, \dots, \mathbf{a}_N)$ sampled from some distribution (e.g., uniform)

but... do we care? How good does the target need to be anyway?

+ dead simple

+ efficiently parallelizable

- not very accurate

very popular as an *improvement* to actor-critic methods in offline RL

sometimes called *sample and rank* or *rejection sampling*

More accurate solution:

- cross-entropy method (CEM)
 - simple iterative stochastic optimization
- CMA-ES
 - substantially less simple iterative stochastic optimization

works OK, for up to about 40 dimensions

Q-learning with continuous actions

Option 2: learn an approximate maximizer

DDPG

“deterministic” actor-critic
= approximate Q-learning

$$\max_{\mathbf{a}} Q_{\phi}(\mathbf{s}, \mathbf{a}) = Q_{\phi}(\mathbf{s}, \arg \max_{\mathbf{a}} Q_{\phi}(\mathbf{s}, \mathbf{a}))$$

idea: train another network $\mu_{\theta}(\mathbf{s})$ such that $\mu_{\theta}(\mathbf{s}) \approx \arg \max_{\mathbf{a}} Q_{\phi}(\mathbf{s}, \mathbf{a})$

how? just solve $\theta \leftarrow \arg \max_{\theta} Q_{\phi}(\mathbf{s}, \mu_{\theta}(\mathbf{s}))$

$$\frac{dQ_{\phi}}{d\theta} = \frac{d\mathbf{a}}{d\theta} \frac{dQ_{\phi}}{d\mathbf{a}}$$

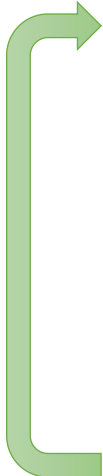
very similar to reparameterized
gradient that we learned about before!

new target $y_j = r_j + \gamma Q_{\phi'}(\mathbf{s}'_j, \mu_{\theta}(\mathbf{s}'_j)) \approx r_j + \gamma Q_{\phi'}(\mathbf{s}'_j, \arg \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j))$

so actor-critic and Q-learning are not that different!

DDPG algorithm summary

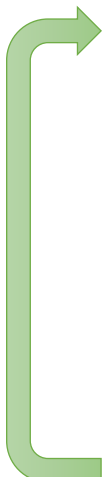
DDPG:

- 
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$, add it to $\mathcal{R}$
 2. sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j\}$ from $\mathcal{R}$ uniformly
 3. compute $y_j = r(\mathbf{s}_j, \mathbf{a}_j) + \gamma Q_{\bar{\phi}}(\mathbf{s}'_j, \mu_{\bar{\theta}}(\mathbf{s}'_j))$ using *target* nets $Q_{\bar{\phi}}$ and $\mu_{\bar{\theta}}$
 4. $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_{\phi}}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_{\phi}(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
 5. $\theta \leftarrow \theta + \beta \sum_j \frac{d\mu}{d\theta}(\mathbf{s}_j) \frac{dQ_{\phi}}{d\mathbf{a}}(\mathbf{s}_j, \mu(\mathbf{s}_j))$
 6. update $\bar{\phi}$ and $\bar{\theta}$ (e.g., Polyak averaging)

can improve with clipped double-Q

$$\mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}'|\mathbf{s}')} [\min_{i \in \{1,2\}} Q_{\bar{\phi}_i}(\mathbf{s}', \mathbf{a}')]]$$

Off-policy actor-critic:

- 
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$, add it to $\mathcal{R}$
 2. sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j\}$ from $\mathcal{R}$ uniformly
 3. compute $y_j = r(\mathbf{s}_j, \mathbf{a}_j) + \gamma \mathbb{E}_{\mathbf{a}'_j \sim \pi_{\bar{\theta}}} [Q_{\bar{\phi}}(\mathbf{s}'_j, \mathbf{a}'_j)]$ using *target* nets $Q_{\bar{\phi}}$ and $\mu_{\bar{\theta}}$
 4. $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_{\phi}}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_{\phi}(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
 5. $\theta \leftarrow \theta + \beta \sum_j \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_j|\mathbf{s}_j) Q_{\phi}(\mathbf{s}_j, \mathbf{a}_j)$
 6. update $\bar{\phi}$ and $\bar{\theta}$ (e.g., Polyak averaging)

What are some other improvements we learned about?

Part 5:

A little bit of theory

Value function learning theory

value iteration algorithm:

- 
1. set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')$
 2. set $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

0.2	0.3	0.4	0.3
0.3	0.3	0.5	0.3
0.4	0.4	0.6	0.4
0.5	0.5	0.7	0.5

does it converge?

and if so, to what?

define an operator $\mathcal{B}$: $\mathcal{B}V = \max_{\mathbf{a}} r_{\mathbf{a}} + \gamma \mathcal{T}_{\mathbf{a}}V$

stacked vector of rewards at all states for action $\mathbf{a}$

matrix of transitions for action $\mathbf{a}$ such that $\mathcal{T}_{\mathbf{a},i,j} = p(\mathbf{s}' = i | \mathbf{s} = j, \mathbf{a})$

V^* is a *fixed point* of $\mathcal{B}$ $V^*(\mathbf{s}) = \max_{\mathbf{a}} r(\mathbf{s}, \mathbf{a}) + \gamma E[V^*(\mathbf{s}')]$, so $V^* = \mathcal{B}V^*$

always exists, is always unique, always corresponds to the optimal policy

...but will we reach it?

Value function learning theory

value iteration algorithm:

- 
1. set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')$
 2. set $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

V^* is a *fixed point* of $\mathcal{B}$ $V^*(\mathbf{s}) = \max_{\mathbf{a}} r(\mathbf{s}, \mathbf{a}) + \gamma E[V^*(\mathbf{s}')]$, so $V^* = \mathcal{B}V^*$

we can prove that value iteration reaches V^* because $\mathcal{B}$ is a *contraction*

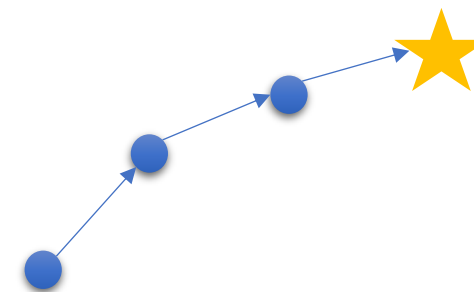
contraction: for any V and $\bar{V}$, we have $\|\mathcal{B}V - \mathcal{B}\bar{V}\|_{\infty} \leq \gamma \|V - \bar{V}\|_{\infty}$

gap always gets smaller by γ !
(with respect to ∞ -norm)

what if we choose V^* as $\bar{V}$? $\mathcal{B}V^* = V^*$!

$$\|\mathcal{B}V - V^*\|_{\infty} \leq \gamma \|V - V^*\|_{\infty}$$

0.2	0.3	0.4	0.3
0.3	0.3	0.5	0.3
0.4	0.4	0.6	0.4
0.5	0.5	0.7	0.5



Non-tabular value function learning

value iteration algorithm (using $\mathcal{B}$):

➡ 1. $V \leftarrow \mathcal{B}V$

fitted value iteration algorithm (using $\mathcal{B}$ and Π):

➡ 1. $V \leftarrow \Pi \mathcal{B}V$

fitted value iteration algorithm:

➡ 1. set $\mathbf{y}_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\phi(\mathbf{s}'_i)])$
2. set $\phi \leftarrow \arg \min_{\phi} \frac{1}{2} \sum_i \|V_\phi(\mathbf{s}_i) - \mathbf{y}_i\|^2$

what does this do?

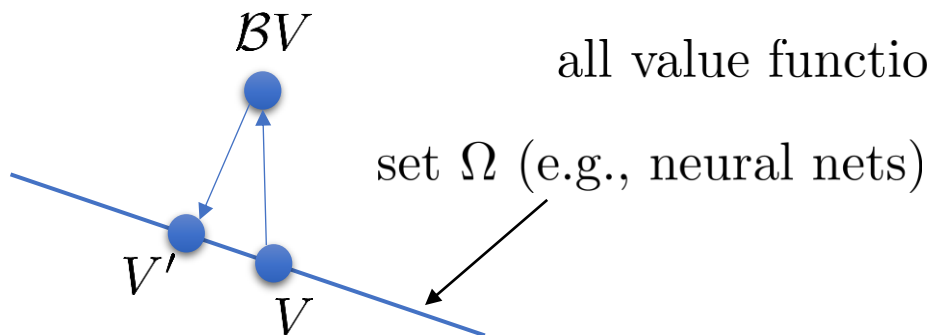
define new operator Π : $\Pi V = \arg \min_{V' \in \Omega} \frac{1}{2} \sum \|V'(\mathbf{s}) - V(\mathbf{s})\|^2$

Π is a *projection* onto Ω (in terms of ℓ_2 norm)

updated value function

$$V' \leftarrow \arg \min_{V' \in \Omega} \frac{1}{2} \sum \|V'(\mathbf{s}) - (\mathcal{B}V)(\mathbf{s})\|^2$$

all value functions represented by, e.g., neural nets



Non-tabular value function learning

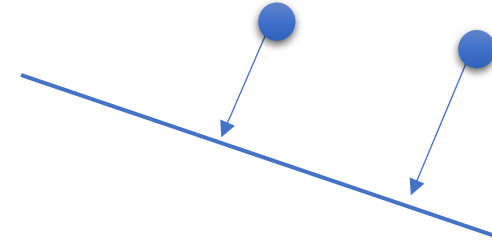
fitted value iteration algorithm (using $\mathcal{B}$ and Π):

➡ 1. $V \leftarrow \Pi \mathcal{B} V$

$\mathcal{B}$ is a contraction w.r.t. ∞ -norm (“max” norm)

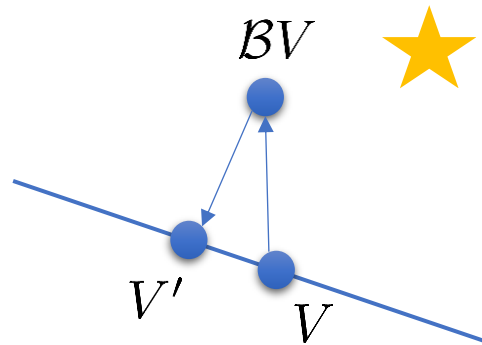
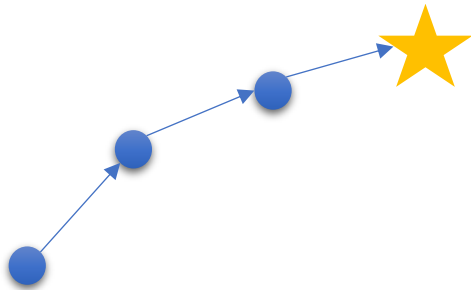
Π is a contraction w.r.t. ℓ_2 -norm (Euclidean distance)

but... $\Pi \mathcal{B}$ is not a contraction of any kind



$$\|\mathcal{B}V - \mathcal{B}\bar{V}\|_{\infty} \leq \gamma \|V - \bar{V}\|_{\infty}$$

$$\|\Pi V - \Pi \bar{V}\|^2 \leq \|V - \bar{V}\|^2$$



Conclusions:
value iteration converges
(tabular case)
fitted value iteration does **not**
converge
not in general
often not in practice

What about fitted Q-iteration?

fitted Q iteration algorithm:

- 
1. set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\phi(\mathbf{s}'_i)]$
 2. set $\phi \leftarrow \arg \min_\phi \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$

define an operator $\mathcal{B}$: $\mathcal{B}Q = r + \gamma \mathcal{T} \max_{\mathbf{a}} Q$

 max now after the transition operator

define an operator Π : $\Pi Q = \arg \min_{Q' \in \Omega} \frac{1}{2} \sum \|Q'(\mathbf{s}, \mathbf{a}) - Q(\mathbf{s}, \mathbf{a})\|^2$

fitted Q-iteration algorithm (using $\mathcal{B}$ and Π):

- 
1. $Q \leftarrow \Pi \mathcal{B} Q$

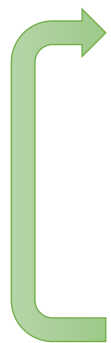
$\mathcal{B}$ is a contraction w.r.t. ∞ -norm (“max” norm)

Π is a contraction w.r.t. ℓ_2 -norm (Euclidean distance)

$\Pi \mathcal{B}$ is not a contraction of any kind **Applies also to online Q-learning**

A sad corollary

batch actor-critic algorithm:

- 
1. sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run it on the robot)
 2. fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
 3. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

An aside regarding terminology

V^π : value function for policy π
this is what the critic does

V^* : value function for optimal policy π^*
this is what value iteration does

ℓ_∞ contraction $\mathcal{B}$ (but without max)


$$y_{i,t} \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1})$$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$



ℓ_2 contraction Π

fitted bootstrapped policy evaluation doesn't converge!

What does this mean?

- Fitted value-based methods (Q-learning, Q-function actor-critic) are often a bit unstable
 - In practice it means that value functions might not converge
 - Explains why more hyperparameter tuning is often needed
- There are tricks that make the problem “less bad”
 - Target networks really help!
 - Replay buffers really help!
- More room for developing new more stable algorithms, and better understanding the existing ones!